

SET6 A1. Как построить минимальный остов?

1. Анализ и эффективная реализация алгоритмов

ALG_1 (Обратный алгоритм Краскала)

Алгоритм реализует стратегию «удаления лишних ребер»: он начинает с полного набора ребер и удаляет самые тяжелые, если это не нарушает связность.

Обоснование эффективной реализации: Проверка связности всего графа после каждого удаления избыточна. Достаточно проверить наличие альтернативного пути между концами удаляемого ребра u и v (игнорируя само ребро). Это реализуется запуском DFS или BFS от u до v и занимает $O(V+E)$ на каждую проверку — оптимально в данном алгоритме.

Сложность:

- Сортировка ребер — $O(E \log E)$
- Цикл по E ребрам, в каждом проверка связности (DFS): $O(V+E)$
- **Итоговая сложность:** $O(E \cdot (V + E)) = O(E^2)$ в худшем случае

Код:

```
using namespace std;
```

```
struct Edge {  
    int u, v, w;  
    bool operator>(const Edge& other) const;  
    bool operator==(const Edge& other) const;  
};
```

```
class Graph {  
    void addEdge(int u, int v, int w); // Добавляет ребро (u, v) с весом w  
    void removeEdge(int u, int v, int w); // Удаляет ребро (u, v) с весом w  
    bool canIgnore(int u, int v, int w); // Проверяет, остается ли граф связным, если временно  
    игнорировать ребро (u, v, w)  
    vector<Edge> getEdges() const; // Возвращает текущее множество ребер графа  
    bool findPath(int u, int v, vector<Edge>& path); // Ищет путь между u и v, заполняет path  
};
```

```
vector<Edge> ALG_1(int V, vector<Edge> E) {  
    sort(E.begin(), E.end(), greater<Edge>()); // Сортировка по невозрастанию весов
```

```
    Graph g(V);  
    for (const auto& e : E) g.addEdge(e.u, e.v, e.w);
```

```
    for (const auto& e : E) {  
        if (g.canIgnore(e.u, e.v, e.w)) {  
            g.removeEdge(e.u, e.v, e.w);  
        }  
    }
```

```
    // Собираем оставшиеся ребра
```

```
    vector<Edge> T = g.getEdges();  
    return T;  
}
```

ALG_2 (Классический алгоритм Краскала с произвольным выбором)

Алгоритм строит дерево «снизу вверх», добавляя по одному ребру. Он перебирает ребра графа в случайном порядке. Для каждого ребра проверяется: если его добавление к уже выбранным ребрам T не создает цикл, то ребро включается в T .

Обоснование эффективной реализации: Самый эффективный способ проверки циклов — структура UNION-FIND (DSU), с оптимизациями сжатия пути и объединения по рангу. Амортизированная сложность операций — $O(\alpha(V))$, что практически константа.

Сложность:

- Проход по всем ребрам: E итераций.
- Операции `find` и `union`: почти константное время $O(\alpha(V))$
- **Итоговая сложность:** $O(E \alpha(V)) \approx O(E)$

Код:

```
using namespace std;
// Edge из предыдущего кода для ALG_1

struct DSU {
public:
    DSU(int n); // Создает n отдельных множеств
    int find(int x); // Находит представителя множества (с путем сжатия)
    void unite(int x, int y); // Объединяет множества, содержащие x и y
};

vector<Edge> ALG_2(vector<Edge> edges, int V) {
    random_device rd;
    mt19937 gen(rd());
    shuffle(edges.begin(), edges.end(), gen);

    DSU dsu(V);
    vector<Edge> T;
    for (const auto& e : edges) {
        if (dsu.find(e.u) != dsu.find(e.v)) {
            dsu.unite(e.u, e.v);
            T.push_back(e);
        }
    }
    return T;
}
```

ALG_3 (Алгоритм удаления самого тяжелого ребра в цикле)

Он поочередно добавляет случайные ребра в набор T . Как только после добавления очередного ребра в T образуется цикл, алгоритм находит в этом цикле ребро с самым большим весом и удаляет его.

Обоснование эффективной реализации: Для каждого ребра сначала выполняется проверка наличия пути, которая позволит избежать лишних операций, между его концами в текущем T без учета добавляемого ребра. Если путь найден — добавление создаст цикл. Поиск пути выполняется с помощью BFS/DFS (гарантировано находит путь за $O(V+E)$), граф хранится в виде списка смежности для быстрого обхода. При обнаружении цикла выполняется линейный проход по найденному пути для определения ребра с максимальным весом.

Сложность:

- Перемешивание — $O(E)$
- Для каждого из E ребер выполняется BFS/DFS — $O(V+E)$ в худшем случае
- **Итоговая сложность:** $O(E \cdot (V + E)) = O(E^2)$ в худшем случае

Код:

```
using namespace std;
// Edge и Graph из предыдущего кода для ALG_1

vector<Edge> ALG_3(int V, vector<Edge> E) {
    random_device rd;
    mt19937 gen(rd());
    shuffle(E.begin(), E.end(), gen);

    Graph g(V);
    vector<Edge> T;

    for (const auto& e : E) {
        vector<Edge> path;

        // Проверяем, есть ли путь между u и v в текущем графе
        if (g.findPath(e.u, e.v, path)) {
            // Нашли цикл, ищем ребро с максимальным весом
            Edge maxEdge = e;
            for (const auto& pe : path) {
                if (pe.w > maxEdge.w) {
                    maxEdge = pe;
                }
            }
            // Если текущее ребро не максимальное — добавляем, удаляем максимальное
            if (!(maxEdge == e)) {
                g.addEdge(e.u, e.v, e.w);
                g.removeEdge(maxEdge.u, maxEdge.v, maxEdge.w);

                auto it = find(T.begin(), T.end(), maxEdge);
                if (it != T.end()) {
                    T.erase(it);
                }
                T.push_back(e);
            }
            // Если текущее ребро максимальное — просто игнорируем
        } else {
            // Нет пути — просто добавляем ребро
            g.addEdge(e.u, e.v, e.w);
            T.push_back(e);
        }
    }
    return T;
}
```

2. Формирует ли алгоритм минимальное остовное дерево (MST)?

ALG_1 — Да, т.к.:

Это «обратный» алгоритм Краскала. Он удаляет ребра, которые не являются «безопасными». Согласно свойству цикла, самое тяжелое ребро любого цикла не может принадлежать MST. Удаляя их по порядку уменьшения веса, мы оставляем только легкие ребра, образующие остов.

ALG_2 — Нет, т.к.:

В задании указано «выбранное случайным образом». Если мы выберем тяжелое ребро первым и оно не образует цикл, оно попадет в T. Пример: Треугольник с весами 1, 10, 100. Если первым выбрать 100, а вторым 10, они не образуют цикл и попадут в T. MST же состоит из ребер 1 и 10.

ALG_3 Да, т.к.:

Этот алгоритм напрямую использует свойство цикла: в любом цикле ребро с максимальным весом не входит в MST. Поскольку алгоритм находит циклы и удаляет именно такие ребра, в итоге останется минимальный остов.